Hotel Management Backend

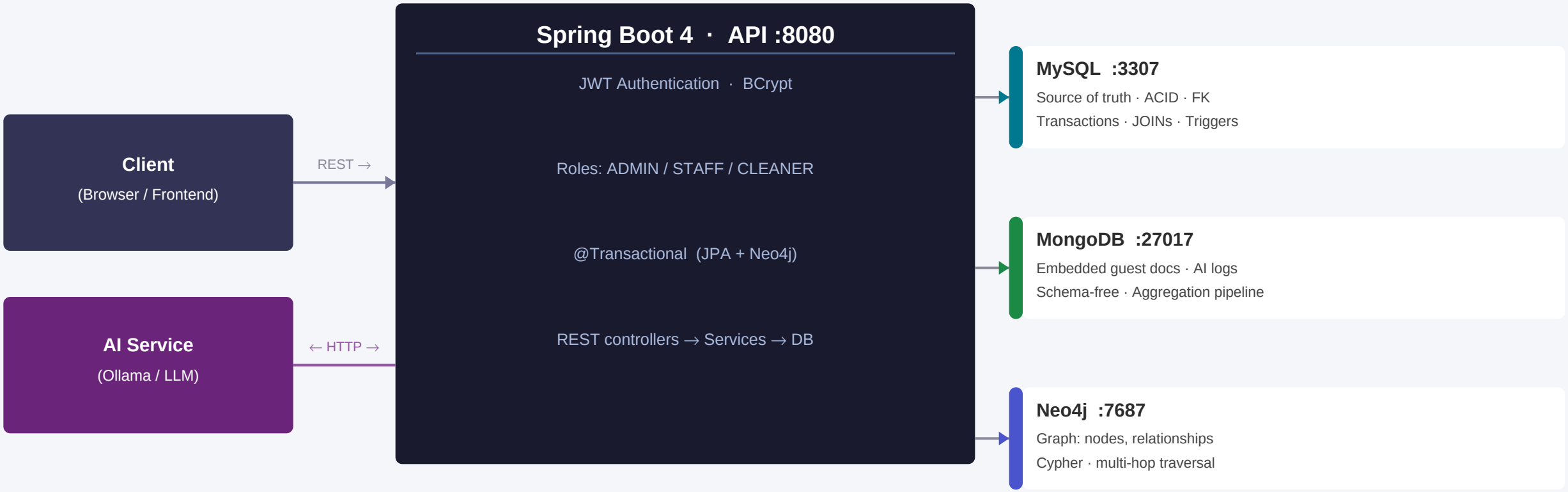
Databases for Developers — Oral Exam Presentation

MySQL :3307 | **MongoDB :27017** | **Neo4j :7687**

Spring Boot 4 · JWT Auth · Docker Compose

1 · System Architecture

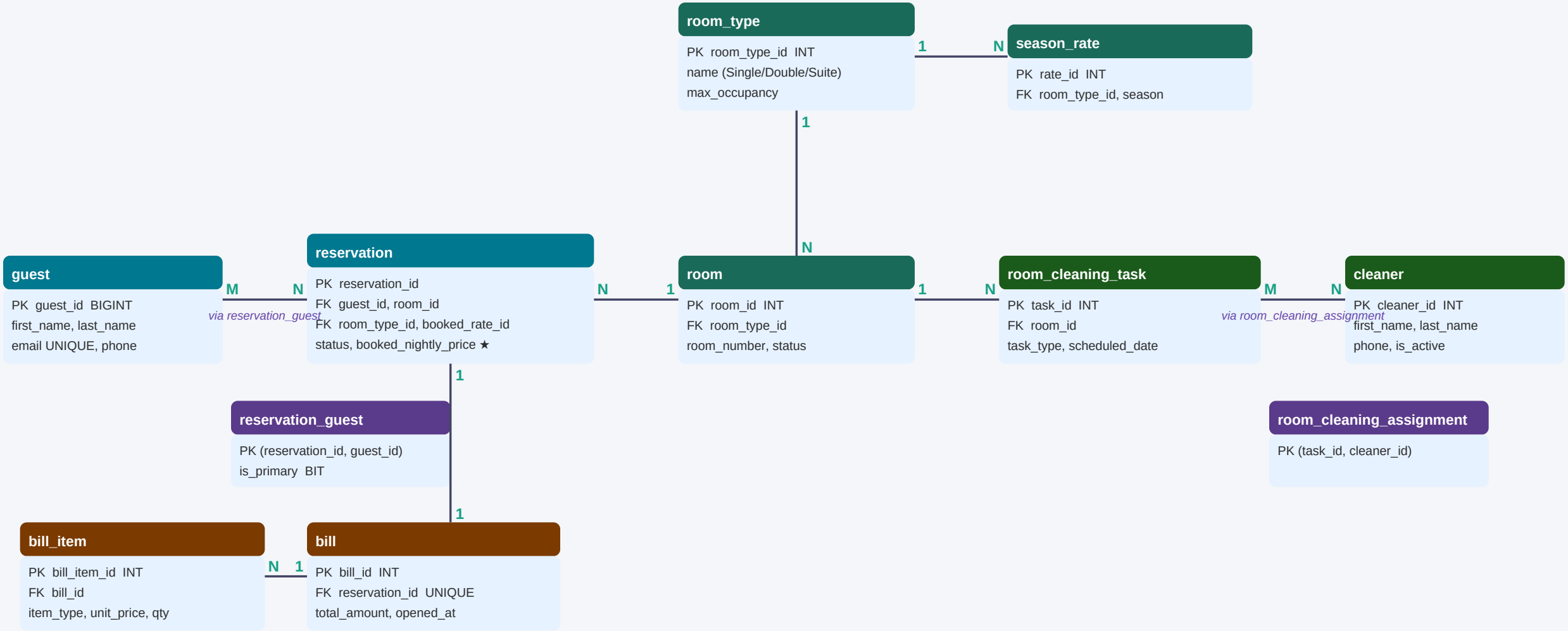
Data flow and responsibilities of each service



POST /api/migrate → Copies MySQL data into MongoDB & Neo4j · Production: CDC (Change Data Capture) + Kafka for real-time sync

2 · Relational Database Design — ERD

Entities, PK/FK, cardinalities · 13 tables · InnoDB (ACID)



★ *booked_nightly_price = historical snapshot (deliberate design — not a 3NF violation)*

3 · Normalization — 1NF → 3NF

How we reached our schema and key design choices

1NF

Atomic values only

Atomic values only

No lists / arrays in one column

Each field = one indivisible value

✓ credit_card_last4: only 4 digits

✓ status: one ENUM value per row

✓ Reservations in own table,

not comma-list in guest row

2NF

Full key dependency

Full key dependency

All non-key fields depend on

the WHOLE primary key

Relevant for COMPOSITE PKs

✓ reservation_guest.is_primary

depends on BOTH key columns

✗ guest_email here → only guest_id

→ moved to guest table

3NF

No transitive dependencies

No transitive dependencies

Non-key fields must not depend

on other non-key fields

A → B → C: move C to own table

✓ room_type_name lives in room_type,

NOT in room or reservation

■ booked_nightly_price = intentional

snapshot — NOT a violation

4 · MongoDB Design

Collections, embedding vs. referencing, document model

guest collection (embedded model)

```
{
  "guestId": 5,
  "firstName": "Magnus",
  "email": "magnus@hotel.dk",
  "reservations": [         <- EMBEDDED
    {
      "reservationId": 42,
      "checkInDate": "2024-06-01",
      "roomType": "Double",
      "bills": [            <- EMBEDDED
        {
          "totalAmount": 2400.00,
          "items": [...]
        }
      ]
    }
  ]
}
```

EMBEDDED — Guest + Reservations + Bills

One document = full guest history

Single query — no JOINs needed

Best when data is always read together

Trade-off: updates must sync with MySQL

REFERENCED — Rooms / Cleaners / Rates

Exist independently of reservations

Shared across many documents

Updated frequently (room status)

Stored as separate collections + IDs

ai_interactions (MongoDB ONLY)

Unstructured LLM chat logs

Schema-free — no fixed columns

Fields: sessionId, userMessage,

aiResponse, model, tokensUsed

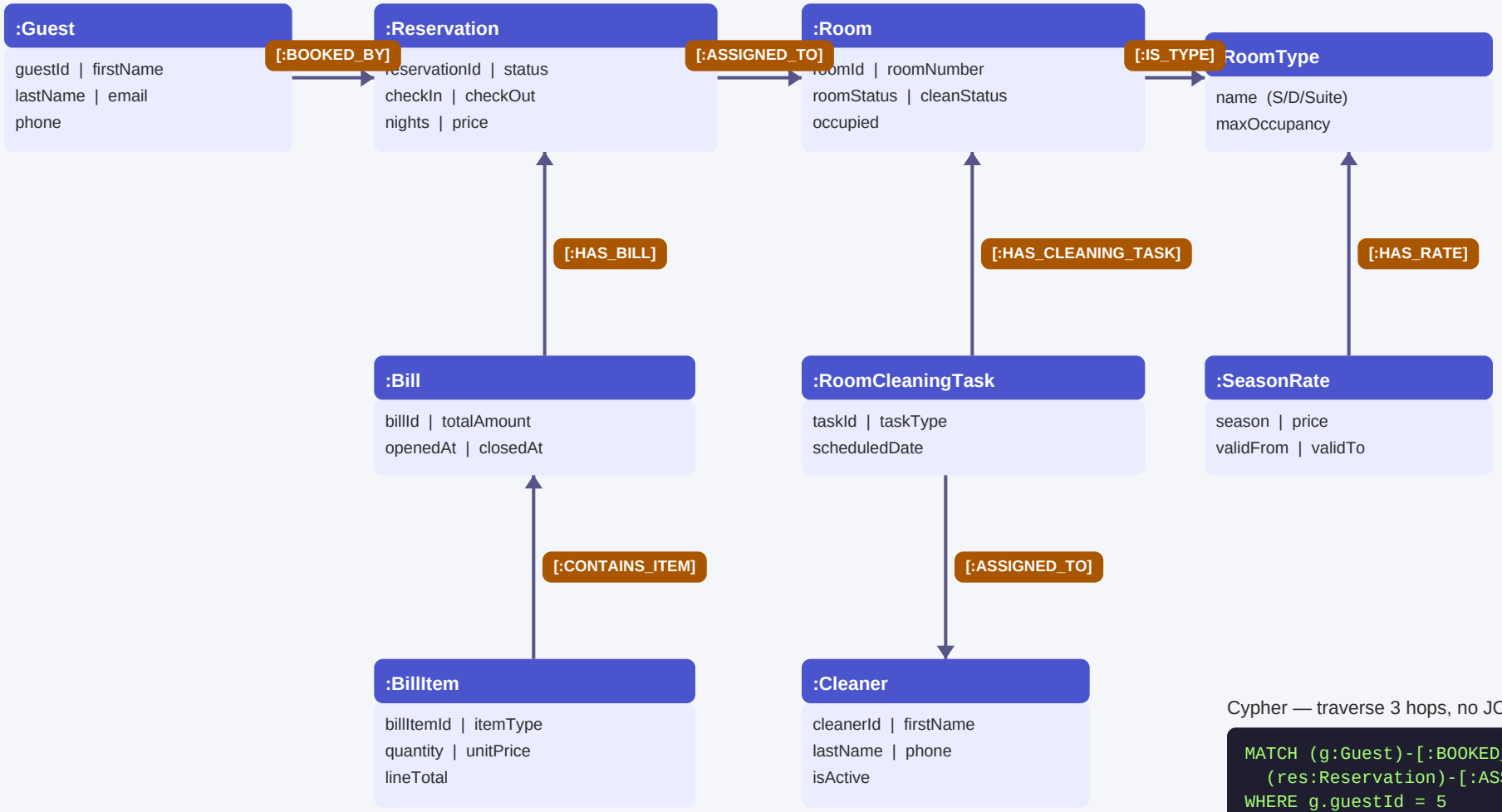
Not a fit for any relational schema

Collections in this project

guests · rooms · cleaners · season_rates · ai_interactions

5 · Neo4j Graph Model

Nodes, relationships, properties — the hotel graph



Cypher — traverse 3 hops, no JOINs

```
MATCH (g:Guest)-[:BOOKED_BY]->
  (res:Reservation)-[:ASSIGNED_TO]->(r:Room)
WHERE g.guestId = 5
RETURN g.firstName, res.status, r.roomNumber
```

6 · Cross-Database Query Comparison

Scenario: find all CONFIRMED reservations for room 101

MySQL (SQL)

```
SELECT g.first_name,
       g.last_name,
       r.reference_no,
       r.check_in_date
FROM reservation r
JOIN guest g
  ON r.guest_id = g.guest_id
JOIN room rm
  ON r.room_id = rm.room_id
WHERE rm.room_number = '101'
  AND r.status = 'CONFIRMED';
```

MongoDB (Aggregation)

```
db.guests.aggregate([
  { $unwind: '$reservations'},
  { $match: {
    'reservations.room': '101',
    'reservations.status':
      'CONFIRMED'
  }},
  { $project: {
    firstName: 1, lastName: 1,
    'reservations.referenceNo':1
  }}
])
```

Neo4j (Cypher)

```
MATCH (g:Guest)
  -[:BOOKED_BY]->
  (res:Reservation)
  -[:ASSIGNED_TO]->
  (r:Room)
WHERE
  r.roomNumber = '101'
  AND res.status = 'CONFIRMED'
RETURN
  g.firstName, g.lastName,
  res.referenceNo
```

MySQL: Explicit JOINs · set-based · ACID-strong · schema-enforced

MongoDB: Pipeline: $unwind → $match → $project · one document fetch

Neo4j: Pattern-match traversal (MATCH) · no JOINs · best for graph hops

7 · Transactions — ACID Guarantees

Concrete example: Guest check-in — 3 operations, one transaction

A

Atomicity

All 3 steps succeed or none at all

C

Consistency

FK constraints ensure no orphan records

I

Isolation

Concurrent bookings: only one wins

D

Durability

Committed data survives server crash

```
BEGIN;

  UPDATE reservation
  SET status = 'CHECKED_IN'
  WHERE reservation_id = 42;        -- step 1

  UPDATE room
  SET occupied = 1,
      room_status = 'OCCUPIED'
  WHERE room_id = 15;               -- step 2

  INSERT INTO bill
    (reservation_id, opened_at, total_amount)
  VALUES (42, NOW(), 0.00);         -- step 3

COMMIT;
-- If anything fails → ROLLBACK
```

@Transactional — Spring Boot

Service methods annotated with

@Transactional

Spring wraps method in

BEGIN ... COMMIT automatically

Any exception triggers

automatic ROLLBACK

Two TX managers configured:

JpaTransactionManager (@Primary)

Neo4jTransactionManager

8 · Indexes & Performance

Key indexes, design decisions, and what we deliberately left out

Index = book's back index — jump directly to the right row instead of scanning all rows → No index: O(n) → B-tree index: O(log n)

Table	Index	Type	Why?
guest	idx_guest_email	B-tree	Login lookup — most frequent query
guest	idx_guest_name (last, first)	Composite	Name search, multi-column ORDER BY
reservation	idx_reservation_dates	Composite	Date-range availability queries
reservation	idx_reservation_status	B-tree	Filter CONFIRMED / CHECKED_IN fast
reservation	idx_reservation_guest	B-tree	FK join: all reservations for a guest
bill	idx_bill_reservation	B-tree	FK join performance on billing
room	idx_room_status	B-tree	Filter AVAILABLE rooms quickly
audit_log	idx_audit_table/op/time	B-tree	Fast audit trail queries

What we did NOT index — and why:

bill_item.description — text search unused, adds write cost with no read benefit

occupied BIT — only 2 values (low cardinality): index won't help, DB scans ~50% of rows anyway

9 · Security & Auditing

JWT, BCrypt, SQL injection prevention, database audit triggers

JWT — JSON Web Token (stateless auth)

1. POST /api/auth/login → server returns signed token
2. Client sends: Authorization: Bearer <token>
3. Server verifies signature — NO database call needed

Token payload: username, role, expiry (HS512 signed)

Stateless: server holds no session state at all

SQL Injection Prevention

UNSAFE: String q = "SELECT * FROM guest WHERE email = '"

+ email + "'" → attacker sends: ' OR '1'='1

SAFE: Spring Data JPA parameterized queries:

findByEmail(String email)

→ SELECT * FROM guest WHERE email = ?

Input is treated as DATA, never as SQL code

BCrypt — one-way password hashing

"admin123" → BCrypt → "$2a$10$N9qo8uLO..."

One-way: cannot reverse hash back to password

Salt built-in: same password → different hash each time

Prevents rainbow table attacks

Database Audit Triggers (05_audit.sql)

Triggers on: reservation, bill, bill_item, guest, room

Each INSERT/UPDATE/DELETE appends a row to audit_log

Stores: old_values (JSON), new_values (JSON),

changed_by (CURRENT_USER()), changed_at

App does NOTHING — database fires trigger automatically

Roles & Access Control

ADMIN — full access (delete, create, view all)

STAFF — read/update reservations, cannot delete

CLEANER — view/update cleaning tasks only

Enforced via Spring Security @PreAuthorize

```
AFTER UPDATE ON reservation FOR EACH ROW
INSERT INTO audit_log(...) VALUES (
  'reservation','UPDATE', NEW.reservation_id,
  JSON_OBJECT('status', OLD.status),
  JSON_OBJECT('status', NEW.status),
  CURRENT_USER());
```

10 · Trade-offs & Reflections

Why three databases — and what we learned

Why MySQL?

- Structured, relational hotel data
- ACID guarantees for payments
- Complex multi-table JOINs & views
- Referential integrity via FK

Why MongoDB?

- Full guest history in one call
- Unstructured AI/LLM chat logs
- Schema-free — add fields freely
- Read-heavy with nested data

Why Neo4j?

- Multi-hop relationship traversal
- Natural: who cleaned what booked by whom
- Graph queries > recursive SQL
- Future: recommendation engine

What we would do differently

Date types: stored dates as String in Neo4j instead of native date/datetime → less efficient, cannot use Cypher date arithmetic

Sync strategy: manual POST /api/migrate → in production: Change Data Capture (CDC) + Kafka for automatic event-driven propagation

Multi-store Spring Boot: JPA + Neo4j TX managers do NOT auto-configure together in Spring Boot 4 → required explicit @Bean for both (JpaTransactionManager @Primary + Neo4jTransactionManager)

Data consistency: MongoDB/Neo4j counts drift from MySQL if migration is not re-run after changes